VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5
Max Marks: 100

Student Name	Registration Number
Rishabh Raj	24BCE10213
Slot	**Date of Submission**
B22	31/03/2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

2. Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: **Python**

3. Project Report

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Python was started by Guido van Rossum in the late 1980s and was released in 1991. At the time, computer programming was mostly done using powerful but difficult-to-understand languages like C and C++. At the same time, scripting languages were popular but were not powerful enough to handle complex tasks.

The basic idea behind Python was to bridge the gap between the two. Guido van Rossum wanted to develop a powerful yet easy-to-use computer language. One of the main reasons for the creation of Python was the need for a computer language that was easy to read and write so that programmers could easily understand each other's work. This is also reflected in the design philosophy of Python, where clean and simple syntax is preferred over complex structures.

Another significant factor in the development of the Python programming language was its openness, as it was made freely available for use, modification, and distribution by anyone interested in using the language. This led to the development of the Python programming language, as developers worldwide contributed to its development, causing the language to grow significantly over the years.

Therefore, the development of the Python programming language can be summarized as a significant development in the history of programming, as the creator of the programming language, Guido van Rossum, had a significant vision for the development of a programming language, as Python was not just created as a programming language, but as a tool for empowering humanity. In my opinion, Python's simplicity is the main reason why it is widely used even today.

A2. The license — what it actually says

The Python programming language is licensed under the Python Software Foundation License. This license is an open-source software license. This means that Python is free to use, distribute, and modify. This license has very few restrictions compared to other licenses like GPL. The Python Software Foundation License allows individuals and organizations to use Python without worrying about legal obligations.

The most important concept related to open-source software licenses is related to the four freedoms of free software. The four freedoms of free software define what an individual is allowed to do with the software. The four freedoms of free software are: an individual is free to run the software for whatever purpose they want. An individual is free to study how the software works and then change it according to their need. An individual is free to redistribute the software. An individual is free to distribute modified versions of the software so that others can also benefit from their modifications. Python software supports these four freedoms of free software.

Another important question is whether it is possible for a company to take the software, modify it, and sell it without having to disclose their modifications. The answer is yes for the Python software. This is because, since it is using a permissive license, companies can use it for proprietary software development without having to disclose their modified source code. This is different from other licenses, like the GNU General Public License, which requires developers to disclose their source code when they modify it. The main difference between the GNU General Public License and licenses like the MIT License or PSF License is their restrictions. The GNU General Public License is a copyleft license because it requires developers to keep their source code open source. On the other hand, licenses like the MIT License or PSF License are permissive because developers can use them for proprietary or open-source software. If I were to choose a license for my own project, I would like it to be a permissive license like the MIT License or PSF License.

Another important term that should be understood here is the difference between ‘free as in free beer’ and ‘free as in freedom.’ ‘Free as in free beer’ refers to the idea that the software can be used without incurring a cost, while ‘free as in freedom’ refers to the idea that the user has the freedom to use the software in whatever way he or she wants. Python can be classified as free in both these senses, which are the main reasons for its popularity worldwide.

I believe permissive licenses like PSF help software grow faster because they encourage wider adoption.

A3. The ethics of open source

Open source is not merely about code; it's a philosophy about the way knowledge and technology should be shared. An important question in this context is whether all software should be open source. Open source, as a term, supports transparency, cooperation, and innovation in a faster way. When the source code of a software program is made publicly available, developers can learn from the source code, enhance it, and even use it for different purposes. For example, the popularity of the Python programming language can be attributed to the fact that thousands of developers are contributing to it and making tools for it.

However, arguments against making all software open source are also valid in certain situations. In some situations, a software program might be dealing with sensitive information, security, or business, and companies might not want to reveal these details publicly. In these situations, making the software proprietary can be a great way for companies to maintain a competitive edge in the market. Thus, even though making software open source can be highly beneficial, it might not be suitable for all software programs.

Another important issue that comes up in relation to ethics is whether it is fair for a company to create a profitable business using open-source software. Many large corporations use open-source software such as Python, Linux, and many other frameworks for their businesses. Even though this speeds up innovation, there are some concerns that have been raised about whether it is fair for a company to use open-source software for their business.

The term “standing on the shoulders of giants” is very relevant in relation to open-source software. This term implies that current developers are standing on the shoulders of giants because they are using the work that has already been developed by other developers in the past. Open-source software has made it easy for developers to stand on the shoulders of giants because they have access to the already developed tools. This actually enhances innovation rather than inhibiting it because it prevents duplication of work.

In my opinion, the use of open source promotes a balance between personal innovation and collective progress. Although not all software needs to be "open source," the existence of "open source" tools such as Python has greatly lowered the threshold for entry into the field of technology.

Part B — Linux Footprint (Unit 2)

B1. Installation of Python on Linux

Python was installed on the Linux system using the Advanced Package Tool (APT), which is the default package management system in Ubuntu. APT allows users to easily install, update, and manage software from official repositories.

To install Python, the following command was used in the terminal:

```
sudo apt install python3 python3-pip -y
```

This command installs the Python interpreter along with pip, which is the package manager used for installing additional Python libraries. The -y flag automatically confirms the installation process without requiring manual input.

The installation process retrieves the required packages from Ubuntu's official repositories and ensures that all dependencies are installed correctly. This method is preferred because it is secure, reliable, and maintained by the open-source community.

After installation, the Python version was verified using the following command:

```
python3 --version
```

This confirmed that Python was successfully installed and ready for use on the system.

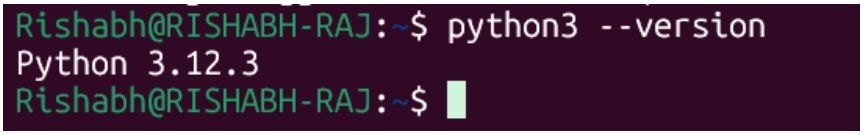

Figure 1-Verification of Python installation using python3 --version command

This installation method demonstrates how open-source software is distributed and maintained through centralized repositories.

B2. Binary Location of Python

After installing Python, the location of its executable binary was identified using standard Linux commands. In Linux systems, binaries are the compiled programs that can be executed directly by the system.

To find the location of the Python binary, the following command was used:

```
which python3
```

This command returns the path of the executable file associated with the python3 command. The output showed that the Python binary is located in the /usr/bin directory.

The /usr/bin directory is a standard location in Linux where user-level executable programs are stored. It contains binaries that are accessible to all users on the system. Since Python is installed as a system-wide package, its binary is placed in this directory so that it can be executed globally from the terminal.

This structure reflects how Linux organizes software efficiently by separating executables, configuration files, and libraries into different directories. Understanding binary locations is important for managing software, debugging issues, and configuring environments.

```
Rishabh@RISHABH-RAJ:~$ which python3
/usr/bin/python3
Rishabh@RISHABH-RAJ:~$
```

Figure 2- Location of Python binary using which python3 command

B3. File Details (Permissions and Ownership)

After identifying the location of the Python binary, its file details were examined to understand its permissions and ownership. In Linux, every file has associated permissions that control who can read, write, or execute it, along with ownership information that specifies which user and group manage the file.

To view these details, the following command was used:

```
ls -l /usr/bin/python3
```

This command displays detailed information about the file, including its permissions, owner, group, size, and last modification date.

The output showed a permission string such as -rwxr-xr-x, which represents different levels of access. The first character indicates the file type, while the next nine characters define permissions for the owner, group, and others. In this case, the owner has full permissions (read, write, execute), while the group and others have read and execute permissions.

```
Rishabh@RISHABH-RAJ:~$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 Nov 12 12:15 /usr/bin/python3 -> python3.12
Rishabh@RISHABH-RAJ:~$
```

Figure 3- File permissions and ownership of Python binary using ls -l command

The ownership of the file is typically assigned to the root user and root group, which means that only the system administrator has full control over the binary. This is important for security because it prevents unauthorized users from modifying critical system files.

Understanding file permissions and ownership is essential in Linux, as it ensures system stability and prevents accidental or malicious changes to important executables like Python. The presence of execute permissions ensures that the binary can be run by all users, while restricted write permissions maintain system security.

B4. Configuration Files of Python

In Linux systems, configuration files store settings that control how software behaves. These files are usually located in the /etc directory, which is reserved for system-wide configuration.

To identify Python-related configuration files, the following command was used:

```
ls /etc | grep python
```

This command lists all files and directories in /etc that are related to Python. The output showed directories such as /etc/python3, which contains configuration settings specific to the Python environment.

The /etc/python3 directory typically includes files that define system-level behaviour, such as default modules, environment settings, and interpreter configurations. These configurations are shared across all users of the system, making them important for maintaining consistency.

```
Rishabh@RISHABH-RAJ:~$ ls /etc | grep python
python3
python3.12
Rishabh@RISHABH-RAJ:~$
```

Figure 4 - Python configuration directory identified using ls /etc | grep python command

The separation of configuration files from binaries and libraries reflects the modular design of Linux. By storing configuration files in /etc, Linux ensures that system settings can be easily managed, modified, or backed up without affecting the actual program files.

Understanding configuration directories is important because it allows administrators and developers to customize software behaviour without modifying the core application itself. This demonstrates how Linux separates configuration from executable files, improving system organization and maintainability.

B5. Python Libraries in Linux

Libraries are an essential part of any software system, as they provide reusable code that programs can use to perform various functions. In Linux, libraries are usually stored in directories such as /usr/lib, where system-wide libraries are maintained.

To identify Python-related libraries, the following command was used:

```
ls /usr/lib | grep python
```

This command lists directories and files in /usr/lib that are associated with Python. The output typically includes directories like /usr/lib/python3, which contain standard libraries required for the Python interpreter to function properly.

These libraries include built-in modules for tasks such as file handling, data processing, networking, and system interaction. They allow developers to perform complex operations without writing code from scratch. For example, Python's standard library provides modules for handling dates, mathematical operations, and input/output processing.

```
Rishabh@RISHABH-RAJ:~$ ls /usr/lib | grep python
python3
python3.12
Rishabh@RISHABH-RAJ:~$
```

Figure 5 - Python libraries located in /usr/lib directory using ls command

The presence of these libraries in a centralized location reflects the modular architecture of Linux. By separating libraries from executables and configuration files, the system ensures better organization, easier updates, and improved maintainability. The availability of a rich standard library is one of the key reasons behind Python's popularity and ease of use.

Understanding library locations is important because it helps in debugging issues, managing dependencies, and extending functionality through additional packages installed via tools like pip.

B6. Log Files in Linux

Log files play an important role in Linux systems by recording system activity, errors, and important events. These logs help administrators and developers monitor system behavior, troubleshoot issues, and analyze performance.

In Linux, most log files are stored in the /var/log directory. To explore the available log files, the following command was used:

```
ls /var/log
```

This command lists all log files and directories present in the system. The output included files such as syslog, auth.log, and dpkg.log, which store different types of system information.

For example, the syslog file contains general system messages and errors generated by the operating system and applications. The auth.log file records authentication-related events such as login attempts, while dpkg.log keeps track of software installation and package management activities.

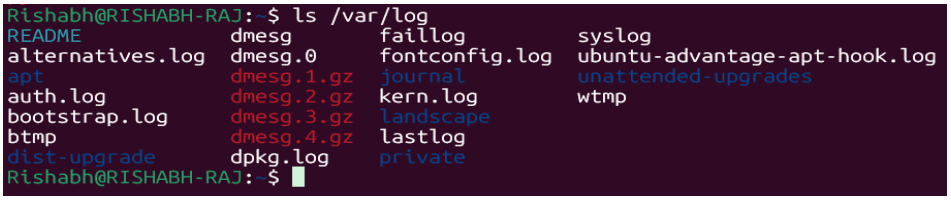

Figure 6 - System log files listed from /var/log directory

These logs are essential for diagnosing problems. In this project, log files were further analyzed using a shell script to count occurrences of errors and extract relevant information. This demonstrates how log files can be used for automated monitoring and analysis.

The structured storage of logs in /var/log reflects the organized design of Linux, where system data is categorized for easy access and maintenance.

The log files were further analyzed using a custom shell script to detect error occurrences, demonstrating practical usage of Linux logs

B7. Users and Permissions

In Linux systems, every process runs under a specific user account, and permissions are used to control access to files and system resources. Understanding users and permissions is essential for maintaining system security and preventing unauthorized access.

Python, when installed using the package manager, runs under the current logged-in user by default. This means that any Python script executed from the terminal runs with the permissions of that user, rather than requiring administrative privileges.

To identify the current user, the following command was used:

```
whoami
```

This command returns the username of the active user. Running Python as a normal user is considered a good security practice because it limits the potential damage that could occur if a script behaves unexpectedly.

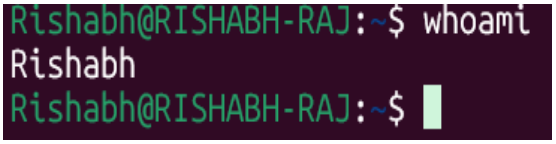

Figure 7 - Current user identified using whoami command

System-level files, such as the Python binary located in /usr/bin, are owned by the root user. This ensures that only the system administrator has permission to modify them, protecting critical components from accidental or malicious changes.

Running applications as a non-root user follows the principle of least privilege, which is a key concept in system security.

The Linux permission system, which includes read, write, and execute permissions for the owner, group, and others, provides fine-grained control over access. This layered security model ensures that software like Python can be used safely by multiple users without compromising the system.

B8. Update Mechanism

In Linux systems, software updates are managed through package management tools such as APT (Advanced Package Tool). This ensures that applications like Python receive regular updates, including security patches, bug fixes, and performance improvements.

Python, when installed through the Ubuntu package manager, is updated using the same system-wide update process. The following commands are used to update the system:

```
sudo apt update
sudo apt upgrade
```

The apt update command refreshes the list of available packages from the repositories, while apt upgrade installs the latest versions of installed software. These updates are maintained by the Ubuntu community and developers, ensuring that the software remains secure and up to date.

One of the key advantages of open-source software is that updates are driven by a global community. Developers continuously identify vulnerabilities and release patches, which are quickly distributed through repositories. This makes the update process reliable and transparent.

In addition to system updates, Python libraries can also be updated using the pip package manager, which allows users to install and manage third-party packages. This dual update system — system-level via APT and library-level via pip — provides flexibility and control over the software environment.

This demonstrates how open-source software benefits from continuous community-driven improvements and rapid security patching.

Overall, the update mechanism in Linux ensures that Python remains secure, stable, and aligned with the latest developments in the open-source ecosystem.

Part C — The FOSS Ecosystem (Units 3 & 4)

1. Dependencies of Python

No open-source software exists independently, and this holds true for Python as well. Python relies on a number of underlying tools and libraries to operate effectively. At the system level, Python depends on tools such as the GNU Compiler Collection (GCC), which is responsible for compiling extensions and system libraries for low-level operations. Python also relies on its own package manager, pip, for handling libraries.

These dependencies enable Python to remain flexible and expandable. For example, Python can be expanded to perform more tasks by installing additional libraries such as NumPy and Pandas.

2. Tools Built on Python

Python has enabled the development of a vast number of tools, frameworks, and applications. Many modern technologies are built on top of Python due to its simplicity and versatility.
Some well-known examples include:

- Django and Flask for web development
- TensorFlow and PyTorch for artificial intelligence and machine learning
- Automation tools and scripting systems used in DevOps

Python is also widely used in real-world applications such as backend systems, data pipelines, and research tools. Its ability to integrate with other technologies makes it a foundational language in many domains.

3. Community and Governance

Python is supported by a large and active global community. The development and management of Python are overseen by the Python Software Foundation, which ensures that the language remains open, stable, and continuously evolving.

The community contributes through:

- Open-source contributions on platforms like GitHub
- Discussions in forums and mailing lists
- Conferences such as PyCon

This collaborative model ensures that Python continues to improve over time and remains relevant in a rapidly changing technological landscape.

4. Relation to LAMP Stack

Traditionally, the LAMP stack refers to a combination of Linux, Apache, MySQL, and PHP. However, Python can be an alternative to PHP in modern web application development.

This gives rise to the following variations:

LAMP → LAPP (Linux, Apache, PostgreSQL, Python)

In this case, Python-based web applications can be created using Python-based web application frameworks like Django and Flask. This shows the seamless integration of Python with existing open-source technologies and the ability to replace existing technologies with more flexible alternatives.

Part D — Open-Source vs Proprietary (Critical Analysis)

Dimension	Python Open Source	MATLAB Proprietary Alternative
Cost	Free to use and distribute	Requires paid license
Security (who can audit the code?)	Source code is open and can be audited by anyone	Source code is closed and controlled by company
Support and reliability	Community support, forums, documentation	Official paid support from company
Freedom to modify	Can modify and redistribute freely	Cannot modify core system
Community vs corporate control	Community-driven development	Controlled by a single organization

Moreover, Python gives the user the freedom to use the software as well as distribute it freely. In addition, Python gives the user the ability to look at the source code. This gives the user more confidence in the software. On the other hand, MATLAB is a proprietary tool owned by a company. This means that the user is restricted by the licensing agreement. In addition, the user cannot look at the source code.

With regard to the cost factor, Python is more favorable compared to MATLAB. This is because Python is free. On the other hand, MATLAB requires a paid license. This makes it difficult for users to acquire the tool.

With regard to the support factor, MATLAB gives the user customer support. This is a great advantage for a business organization. However, Python has a large community of users worldwide. This means that Python gives the user a lot of support.

In terms of flexibility, Python is definitely ahead of MATLAB. Python has the potential to be used in various fields like web development, data science, artificial intelligence, etc. MATLAB is only used for numerical computing and programming.

On the other hand, Python can be considered a more flexible and accessible option compared to MATLAB. This is particularly the case for general-purpose programming and modern applications.

In conclusion, I think Python is a more viable option for deployment compared to MATLAB for various applications. This is mainly attributed to the cost-effectiveness and scalability offered by Python. Although MATLAB may be considered a better option for certain applications within the academic and engineering fields, Python can be regarded as a more future-oriented option. With the given opportunity, I would not only use Python but also contribute to the Python ecosystem by creating and sharing open-source tools.

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

1. Objective

The objective of this script is to display basic system information such as kernel version, logged-in user, system uptime, and Linux distribution details.

2. Script Code

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Rishabh Raj

STUDENT_NAME="Rishabh Raj"
SOFTWARE_CHOICE="Python"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
```

```
DATE=$(date)

DISTRO=$(lsb_release -d | cut -f2)

echo "========================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================"
echo "Software : $SOFTWARE_CHOICE"
echo "Distro   : $DISTRO"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Uptime   : $UPTIME"
echo "Date     : $DATE"
echo ""
echo "License  : Linux is licensed under GPL (Free Software License)"
```

3. Explanation

This script uses variables and command substitution to collect system information. Commands like uname, whoami, and uptime are used to retrieve system details. The output is formatted using echo statements to create a structured display.

4. Output Screenshot

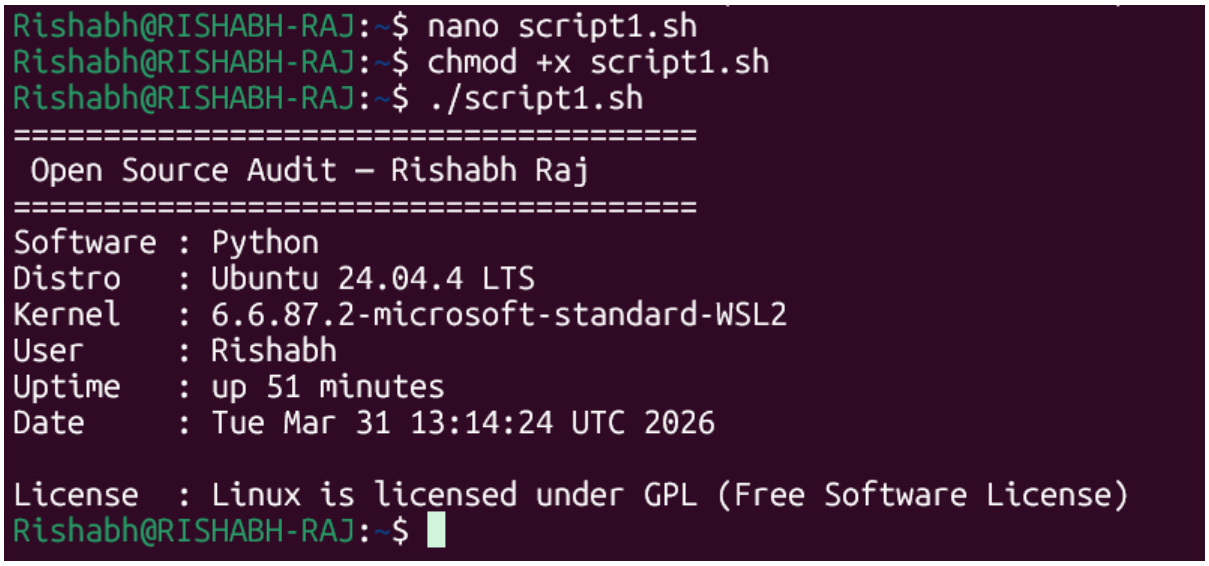

Figure 8 - Output of System Identity Report script

Script 2 — FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

1. Objective

To check whether a specific open-source package (Python) is installed and display its version and details.

2. Script Code

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Rishabh Raj

PACKAGE="python3"

echo "========================================"
echo " FOSS Package Inspector"
echo "========================================"

# Check if package is installed
if dpkg -l | grep -qw $PACKAGE; then
    echo "$PACKAGE is installed."

    echo ""
    echo "Package Details:"
    apt show $PACKAGE 2>/dev/null | grep -E 'Version|Maintainer|Description'

else
    echo "$PACKAGE is NOT installed."
fi

echo ""
echo "Philosophy Note:"

# Case statement
case $PACKAGE in
    python3)
        echo "Python: a powerful open-source language designed for readabil>
        ;;
    git)
        echo "Git: a distributed version control system that enables collab>
        ;;
    mysql)
        echo "MySQL: open-source database powering millions of applications>
        ;;
    apache2)
        echo "Apache: the web server that helped build the modern internet."
        ;;
    *)
      echo "Unknown package"
        ;;
esac
```

3. Explanation

This script uses conditional statements to check if the package is installed using dpkg. It uses grep to filter relevant information and a case statement to display a short description of the software.

4. Output Screenshot

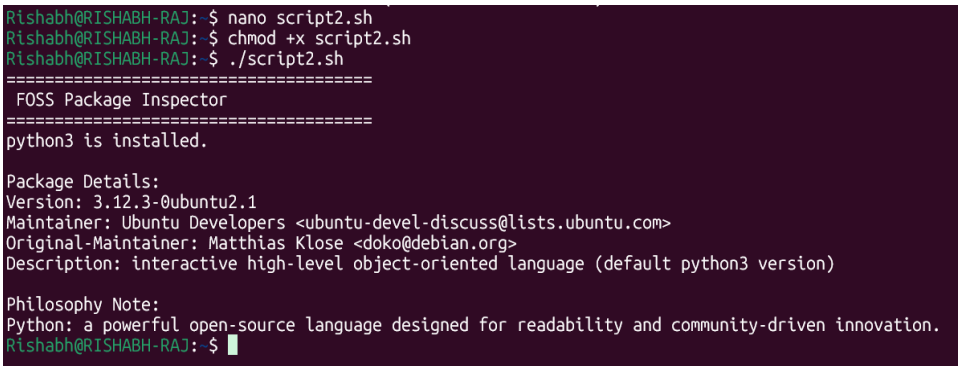

Figure 9 - Output of FOSS Package Inspector script

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

1. Objective

To analyze important system directories and display their disk usage, ownership, and permissions.

2. Script Code

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Rishabh Raj

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "======================================"
echo " Directory Audit Report"
echo "======================================"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3,
$4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

        echo "$DIR => Permissions: $PERMS | Size:
$SIZE"
    else
        echo "$DIR does not exist"
    fi
done

echo ""
echo "======================================"
echo " Python Configuration Check"
echo "======================================"

# Check Python config directory
if [ -d "/etc/python3" ]; then
    PERMS=$(ls -ld /etc/python3 | awk '{print $1, $3,
$4}')
    echo "/etc/python3 exists => Permissions: $PERMS"
else
    echo "/etc/python3 does not exist"
fi
```

3. Explanation

This script uses a for loop to iterate through multiple directories. It uses ls -ld to extract permissions and du -sh to calculate directory size. The script also checks the Python configuration directory.

Output Screenshot

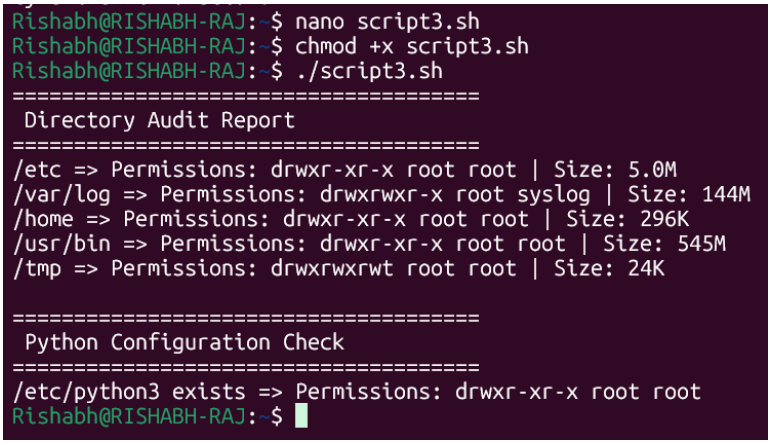

Figure 10 - Output of Disk and Permission Auditor script

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

1. Objective

To analyze a log file and count occurrences of a specific keyword such as "error".

2. Script Code

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Rishabh Raj

LOGFILE=$1
KEYWORD=${2:-"error"}   # default keyword =
error
COUNT=0

echo "======================================"
echo " Log File Analyzer"
echo "======================================"

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Check if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: File is empty.
Retrying..."
    sleep 2
fi

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD";
then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo ""
echo "Keyword '$KEYWORD' found $COUNT times
in $LOGFILE"

echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

3. Explanation

This script reads a log file line by line using a while loop. It checks for a keyword using grep and counts occurrences using a counter variable. It also displays the last few matching lines.

4. Output Screenshot

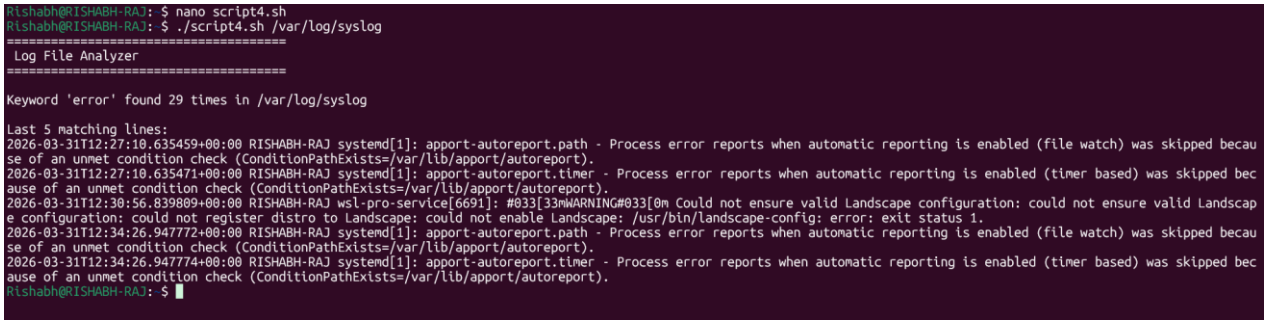

Figure 11 - Output of Log File Analyzer script

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalized open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file

1. Objective

To generate a personalized open-source manifesto based on user input.

2. Script Code

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Rishabh Raj

echo "==================================="
echo " Open Source Manifesto Generator"
echo "==================================="
echo ""

# Take user input
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
USER_NAME=$(whoami)
OUTPUT="manifesto_$USER_NAME.txt"

echo ""
echo "Generating your manifesto..."

# Create manifesto
echo "----------------------------------------" > $OUTPUT
echo " Open Source Manifesto" >> $OUTPUT
echo "----------------------------------------" >> $OUTPUT
echo "Date: $DATE" >> $OUTPUT
echo "" >> $OUTPUT
echo "I use $TOOL as part of my daily work. To me, freedom means $FREEDOM." >> $OUTPUT
echo "I believe in sharing knowledge and building tools that benefit everyone." >> $OUTPUT
echo "One day, I will create $BUILD and make it freely available to the world." >> $OUTPUT
echo "" >> $OUTPUT
echo "— $USER_NAME" >> $OUTPUT

echo ""
echo "Manifesto saved to $OUTPUT"
echo ""
cat $OUTPUT
```

3. Explanation

This script uses the read command to take user input and combines it into a formatted paragraph. The output is saved to a text file using file redirection.

4. Output Screen Shot

```
Rishabh@RISHABH-RAJ:~$ nano script5.sh
Rishabh@RISHABH-RAJ:~$ chmod +x script5.sh
Rishabh@RISHABH-RAJ:~$ ./script5.sh
=====================================
 Open Source Manifesto Generator
=====================================

1. Name one open-source tool you use every day: Linux
2. In one word, what does 'freedom' mean to you? Control
3. Name one thing you would build and share freely: a secure open-source operating system

Generating your manifesto...

Manifesto saved to manifesto_Rishabh.txt

---------------------------------------
 Open Source Manifesto
---------------------------------------
Date: 31 March 2026

I use Linux as part of my daily work. To me, freedom means Control.
I believe in sharing knowledge and building tools that benefit everyone.
One day, I will create a secure open-source operating system and make it freely available to the world.

— Rishabh
Rishabh@RISHABH-RAJ:~$
```

Figure 12 - Output of Open Source Manifesto Generator script